Proyecto Integrador

Samantha Chamorro

Jeniffer Condor

Luis Quisnancela

Programación Orientada a Objetos

Universidad Internacional Del Ecuador

Facultad de Ciencias Técnicas

Ingeniería en Sistemas de la Información

María del Carmen Salazar Torres

11 de octubre del 2026

Fase 1: Proyecto integrador

1. Contexto:

La empresa abrió una segunda sede y necesita que ambas se comuniquen de forma segura. Sin un sistema interno, los equipos podrían terminar usando WhatsApp, correos personales o llamadas. Así, la información quedaría repartida en dispositivos personales y no habría un registro de las decisiones que se toman entre las dos sedes. Además, los mensajes circularían sin control de quién los ve, y personas que ya no trabajan en la empresa podrían seguir con acceso a datos sensibles. Un sistema de correos internos resuelve esto con un único canal de comunicación, con usuarios registrados y acceso protegido por autenticación, de modo que los mensajes queden guardados y solo los vea el personal autorizado. La Ley Orgánica de Protección de Datos Personales tiene por objeto garantizar el derecho a la protección de datos personales ***(Asamblea Nacional del Ecuador, 2021, art. 1).***

2. Selección Justificada de la Aplicación:

Seleccionamos la opción de **Gestión de correos internos** (mensajería entre usuarios registrados, con bandeja de entrada, enviados, lectura y eliminación de mensajes). No se requiere integración con un servicio SMTP externo, ya que el sistema opera como un canal centralizado que resuelve la falta de comunicación interna y de trazabilidad entre las dos sedes.

Elegimos esta opción porque su modelo de datos estructurado es manejable para un primer proyecto en Go y permite operaciones cliente-servidor reales y específicas, como: el cliente envía un mensaje, el servidor lo procesa y almacena de manera segura, y luego el cliente consulta su bandeja de entrada o sus mensajes enviados, y solicita leer o eliminar un mensaje. Descartamos el chat empresarial, la gestión de imágenes y la de videos porque su modelo de datos es más complejo de manejar en esta etapa.

Asimismo, el sistema genera datos fáciles de medir y útiles para el componente estadístico, como el número total de correos almacenados, el volumen de correos enviados por cada usuario y los tiempos de respuesta del servidor.

3. Actores y Usuarios Iniciales:

Remitente:

Es el usuario autenticado que genera y transmite la comunicación operativa dentro del sistema cliente-servidor en Go. Su función principal es redactar mensajes dirigidos a otros miembros de la organización y enviarlos a través de la red para que queden registrados en el servidor. Asimismo, consulta su bandeja de enviados.

Destinatario:

Es el usuario que recibe las comunicaciones enviadas por sus compañeros en el sistema. Su función consiste en consultar su bandeja de entrada personalizada, visualizar los mensajes recibidos y proceder a su lectura, revisión o eliminación. Este rol asegura que la información fluya de manera ordenada y estructurada entre los miembros de las distintas sedes de la empresa.

Administrador:

El administrador es el actor con privilegios exclusivos de gestión técnica y control de accesos sobre la plataforma, que el remitente y el destinatario no pueden realizar. A diferencia de un usuario común, el administrador tiene la capacidad de registrar nuevos usuarios en el sistema, gestionar las credenciales y accesos corporativos, y supervisar el funcionamiento técnico de la plataforma. No puede leer el contenido de los mensajes, lo que garantiza la privacidad y la confidencialidad de la comunicación corporativa.

Usuarios iniciales:

Un administrador y varios usuarios de prueba de cada sede, para validar el funcionamiento del sistema.

Servicio de autenticación (FastAPI):

Este servicio actúa como el núcleo de seguridad de la plataforma, valida las credenciales de los usuarios de forma centralizada y emite las sesiones válidas de acceso requeridas para interactuar con el sistema en Go, garantizando que solo personal verificado acceda al sistema.

Equipo de TI de la empresa:

Es el personal técnico responsable de dar mantenimiento preventivo y correctivo al sistema y de supervisar la infraestructura de red en ambas sedes.

Dirección de la empresa:

Es el rol directivo y de toma de decisiones que requiere que la comunicación interna entre las sedes se mantenga de forma ordenada, segura y plenamente trazable.

Actor	Qué hace	Qué no puede hacer
Remitente	Redacta y envía mensajes, consulta su bandeja de enviados	Administrar usuarios ni leer mensajes ajenos
Destinatario	Consulta su bandeja de entrada, lee y elimina mensajes	Administrar usuarios ni leer mensajes ajenos
Administrador	Registra usuarios, gestiona accesos y supervisa el funcionamiento técnico	Leer el contenido de los mensajes
Servicio de Autenticación	Valida credenciales y emite la sesión ***(FastAPI, s. f.).***	Acceder a los mensajes
Equipo de TI	Da mantenimiento al sistema y supervisa la red de ambas sedes	Leer el contenido de los mensajes
Dirección	Define la necesidad de una comunicación ordenada, segura y trazable	Operar el sistema

4. Matriz de Roles

Rol	Responsabilidades	Integrantes
Coordinación y documentación	Organiza el cronograma y el tablero, redacta los documentos, revisa que todo se entregue a tiempo, lleva la declaración de IA y el análisis ético	Samantha, Jeniffer
Desarrollo de la aplicación	Programa el cliente y el servidor en Go, hace el diagrama de clases y las pruebas funcionales	Luis, Jeniffer
Infraestructura y seguridad	Prepara la red de las dos sedes, la integración con el servicio de autenticación y las pruebas de seguridad	Samantha, Luis

5. Cronograma de Trabajo

Diagrama de Gantt

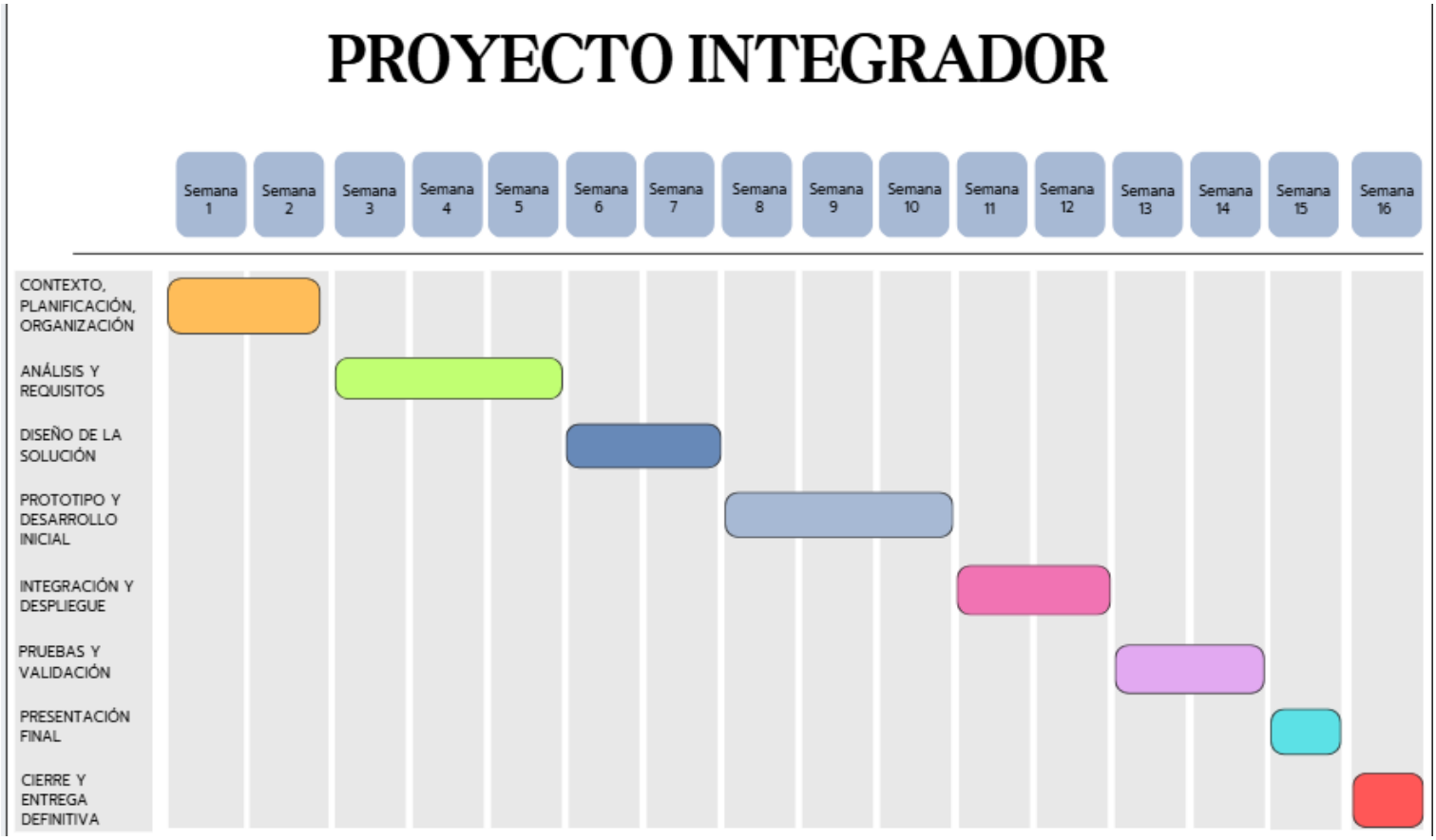


Fig. 1 Diagrama de Gantt con la distribución de las fases

Fases del Proyecto:

Fase 1 (Semanas 1–2): Contexto, planificación y organización. Conocimos el proyecto integrador, formamos nuestro grupo de trabajo y definimos los roles de cada integrante. Seleccionamos y justificamos la aplicación que vamos a desarrollar en Go: Gestión de correos internos, porque su modelo de datos es manejable y nos permite generar datos medibles, y creamos el repositorio Git inicial con su README y su licencia.

Fase 2 (Semanas 3–5): Análisis y requisitos. Identificaremos los actores del sistema (remitente, destinatario, administrador) y redactaremos los requisitos funcionales y no funcionales del módulo (RF-SW-01, RF-SW-02, RF-SW-03) con sus criterios de aceptación. Elaboraremos los casos de uso principales: enviar, leer, eliminar correos y consultar la bandeja de entrada.

Fase 3 (Semanas 6–7): Diseño de la solución. Diseñaremos la arquitectura cliente-servidor de nuestra aplicación en Go, el diagrama de clases y el modelo de datos de mensajes y usuarios. Definiremos también el flujo de comunicación con el servicio de autenticación (FastAPI) y elaboraremos un prototipo visual de la bandeja de entrada.

Fase 4 (Semanas 8–10): Prototipo y desarrollo inicial. Desarrollaremos la estructura modular de nuestra aplicación (servidor, cliente y operaciones principales), siguiendo la organización recomendada del repositorio (app-go/cmd, app-go/internal). Al cierre de esta fase tendremos un prototipo funcional capaz de enviar, leer y eliminar correos, con sus primeras validaciones de datos.

Fase 5 (Semanas 11–12): Integración y despliegue. En la semana 11, una vez terminado el prototipo, iniciaremos la integración de nuestra aplicación Go con el servicio de autenticación FastAPI, para que el acceso a los correos exija una sesión válida. Haremos pruebas de integración, corregiremos errores de interoperabilidad y registraremos las primeras métricas de tiempo de respuesta para el componente estadístico.

Fase 6 (Semanas 13–14): Pruebas y validación. Ejecutaremos pruebas funcionales de las operaciones principales y pruebas con credenciales válidas e inválidas, verificando que el acceso a la bandeja de entrada quede correctamente protegido. Los resultados alimentarán nuestra matriz de cumplimiento de requisitos y el análisis estadístico del proyecto.

Fase 7 (Semana 15): Presentación final. Repasaremos todo el trabajo realizado y haremos pruebas de funcionamiento y operación. Después haremos la demostración en vivo de nuestra aplicación, explicaremos su arquitectura y el flujo de autenticación, y presentaremos los resultados de las pruebas. Defenderemos ante los docentes las decisiones técnicas del proyecto.

Fase 8 (Semana 16): Cierre y entrega definitiva. Después de la defensa, corregiremos los errores y observaciones que nos indiquen los docentes. Actualizaremos el documento final y el repositorio, y entregaremos los manuales de instalación y ejecución junto con los scripts y datos del proyecto. Para cerrar, haremos una retrospectiva con nuestros logros, limitaciones y trabajo futuro, y completaremos la autoevaluación y la coevaluación del equipo.

6. Contrato de comunicación del equipo

6.1 Propósito

El presente contrato establece las normas de comunicación, coordinación y resolución de conflictos que regirán el trabajo del equipo durante el desarrollo del proyecto integrador. Su finalidad es asegurar que la información fluya de manera oportuna y transparente, que cada integrante conozca sus responsabilidades y que las decisiones se tomen de forma justa y

verificable. Al formar parte del equipo, cada integrante se compromete a respetar y cumplir lo aquí acordado.

6.2 Canales de comunicación

Comunicación diaria (WhatsApp). Este canal se utilizará para el intercambio cotidiano de información, la emisión de avisos, el envío de recordatorios y la realización de encuestas. Se eligió porque todos los integrantes lo usan de forma habitual, lo que garantiza una lectura rápida de los mensajes, y porque conserva un historial que sirve como evidencia de los acuerdos y del trabajo realizado.

Reuniones de trabajo (Discord o Microsoft Teams). Estas plataformas se emplearán en sesiones sincrónicas, como la revisión de avances, el diseño de clases y diagramas UML, la depuración conjunta del código y la planificación de entregas. La elección entre una u otra se acordará previamente en el chat, según la disponibilidad del equipo y las necesidades de la sesión (compartir pantalla, grabación, entre otras).

Comunicación presencial (en clase). Los integrantes podrán conversar sobre el proyecto durante las clases. No obstante, todo acuerdo, decisión o tarea asignada de manera presencial deberá registrarse por escrito en el chat de WhatsApp el mismo día. Con ello se evita que algún integrante quede sin información y se garantiza la trazabilidad de lo acordado. Un acuerdo verbal que no haya sido registrado no se considerará válido.

6.3 Tiempos de respuesta

Para que el trabajo no se detenga por falta de comunicación, el equipo adopta los siguientes plazos máximos de respuesta:

Tipo de mensaje	Tiempo máximo de respuesta
Ordinario: consultas, avisos generales, dudas y recordatorios	12 horas
Urgente: entrega próxima, bloqueo en el desarrollo o decisión que no puede esperar	4 horas

Adicionalmente, se establecen las siguientes reglas de funcionamiento:

- **Identificación de urgencias:** los mensajes urgentes deberán marcarse claramente, por ejemplo anteponiendo la palabra «URGENTE», para que sean atendidos con prioridad.

- **Programación de reuniones:** primero se propondrá una hora en el chat y se confirmará con todos los integrantes. En la reunión se tratarán los temas pendientes y se definirán las tareas siguientes.
- **Indisponibilidad:** si un integrante prevé no estar disponible dentro de los plazos establecidos (por salud, viaje, exámenes u otra causa justificada), deberá avisar con anticipación para reorganizar las tareas.
- **Calidad de la respuesta:** responder implica confirmar la lectura y dar una contestación útil; una simple reacción al mensaje no se considera respuesta.

6.4 Mecanismo de toma de decisiones

Las decisiones del equipo se tomarán de manera democrática, mediante votación, con el fin de que todas las opiniones sean escuchadas y consideradas. El procedimiento será el siguiente:

1. **Planteamiento.** Se expone la propuesta o el problema, junto con sus alternativas y argumentos, ya sea por el chat o en una reunión.
2. **Votación.** Se realizará mediante una encuesta de WhatsApp o preguntando de forma directa en la reunión si todos están de acuerdo.
3. **Resolución.** Se adoptará la opción que obtenga la mayoría de los votos.
4. **Registro.** La decisión final y el resultado de la votación se escribirán en el chat para dejar constancia.

Una vez adoptada una decisión por mayoría, todos los integrantes, incluso quienes votaron en contra, se comprometen a respaldarla y a trabajar conforme a ella. En las decisiones de carácter técnico (por ejemplo, la estructura de clases, el uso de herencia o interfaces, o las herramientas de desarrollo), cada postura deberá sustentarse con argumentos antes de la votación.

6.5 Responsabilidades individuales

Cada integrante cumplirá las tareas que se le asignen dentro de los plazos establecidos. La distribución de actividades se acordará en equipo, considerando las capacidades de cada miembro y procurando una carga de trabajo equitativa. Cada persona deberá informar oportunamente sobre su avance y, si enfrenta dificultades, solicitar apoyo antes de que venza el plazo.

6.6 Manejo de conflictos e incumplimientos

Ante el incumplimiento de una tarea o compromiso, el equipo seguirá un procedimiento progresivo que prioriza el diálogo antes de aplicar medidas mayores:

1. **Recordatorio.** Se le recordará al integrante la tarea pendiente por el chat y se conversará con él o ella para conocer las razones del retraso.
2. **Plazo de recuperación.** Se otorgará un plazo de 24 horas para ponerse al día con la actividad pendiente.
3. **Reasignación.** Si el incumplimiento persiste una vez vencido el plazo, la tarea será asignada a otro integrante del equipo para no afectar el cronograma del proyecto.
4. **Escalamiento.** La situación se comunicará a los profesores, para que estén al tanto y orienten al equipo sobre cómo proceder.

Los desacuerdos entre integrantes se tratarán con respeto, centrándose en el problema y no en las personas. Las críticas serán constructivas y se buscarán soluciones que beneficien al equipo en su conjunto.

Repositorio: https://github.com/sammy1208-g/Proyecto_Integrador_3_nivel

7. Diagrama de Clases UML

El siguiente diagrama presenta la versión preliminar de las clases del sistema de correos internos. Se compone de siete clases y una interfaz, separadas en la parte del cliente y la parte del servidor. Las clases se implementarán como structs de Go, y se relacionan por composición y por la interfaz ServicioCorreo, sin usar herencia. Esta versión se completará en la Fase 3.

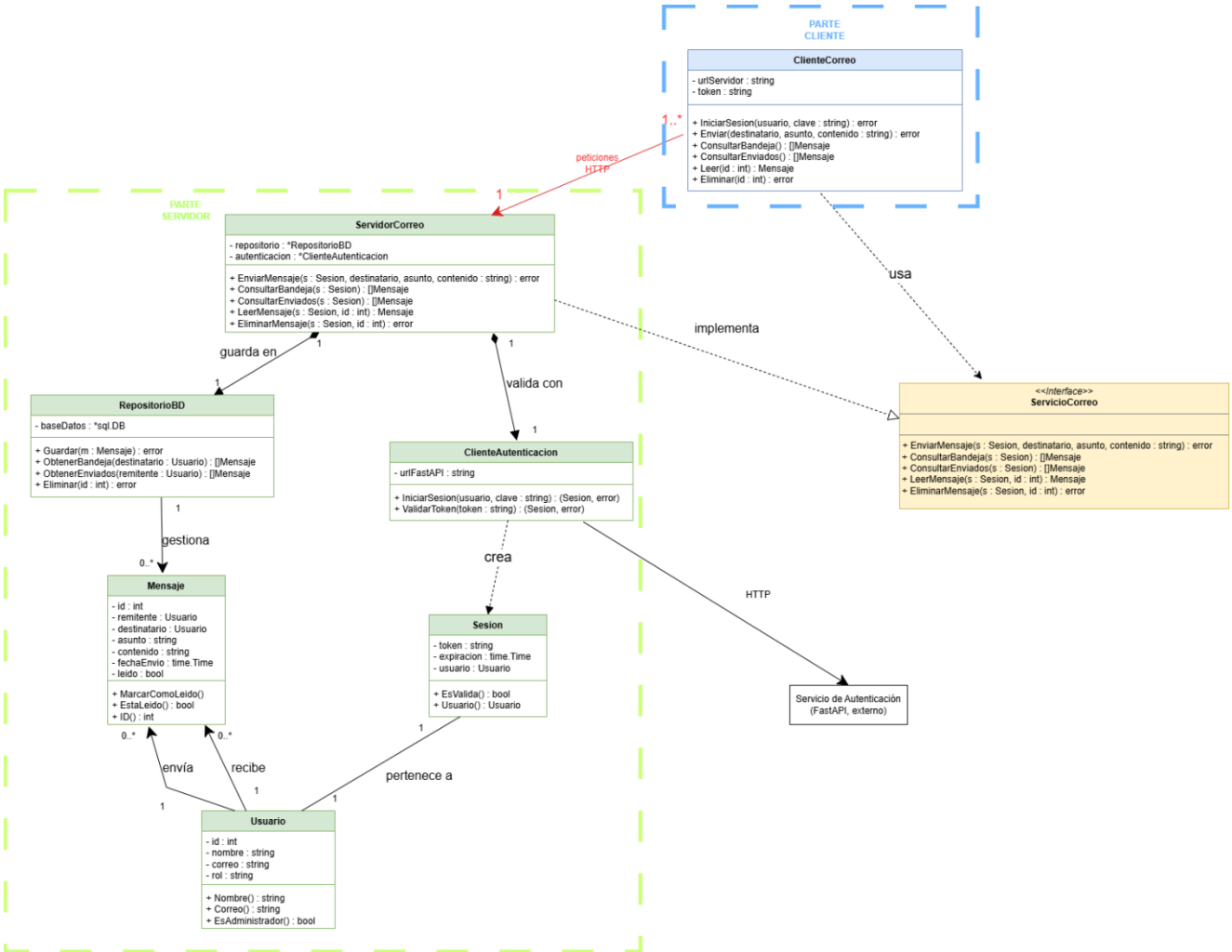


Figura 2. Diagrama de clases UML preliminar del sistema de correos internos.

8. Declaración de integridad académica, licencia y uso de IA

8.1 Integridad académica

Declaramos que este trabajo es producto del esfuerzo del equipo y que las ideas, textos o recursos provenientes de otras fuentes serán citados correctamente. Nos comprometemos a cumplir las normas de integridad académica de la universidad durante todo el desarrollo del proyecto.

8.2 Licencia

El código del proyecto se publicará en el repositorio Git creado en esta fase, bajo la licencia MIT (preliminar) ***(Open Source Initiative, s. f.).*** Esta licencia permite usar, copiar y modificar el código, siempre que se mantenga el aviso de autoría. El repositorio ya fue creado y contiene el README, la licencia y el tablero de seguimiento.

8.3 Uso de inteligencia artificial

Utilizamos Claude (Anthropic) como apoyo para los siguientes fines:

- Orientar la organización del proyecto;
- Revisar la redacción y ordenar el documento;
- Explicar conceptos que resultaban ambiguos;
- Configurar correctamente el tablero de trabajo en el repositorio.

Las decisiones sobre la selección del proyecto, los actores y usuarios, y los demás detalles fueron tomadas por el equipo. Asimismo, revisamos todo el contenido antes de entregarlo.

9. Referencias:

Asamblea Nacional del Ecuador. (2021). *Ley Orgánica de Protección de Datos Personales* (Quinto Suplemento del Registro Oficial n.° 459, 26 de mayo de 2021). https://www.gob.ec/regulaciones/ley-organica-proteccion-datos-personales-0

FastAPI. (s. f.). *OAuth2 with Password (and hashing), Bearer with JWT tokens*. Recuperado el 9 de octubre de 2026, de https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/

GitHub. (s. f.). *Planning and tracking with Projects*. GitHub Docs. Recuperado el 9 de octubre de 2026, de https://docs.github.com/en/issues/planning-and-tracking-with-projects

Open Source Initiative. (s. f.). *The MIT License*. Recuperado el 9 de octubre de 2026, de https://opensource.org/license/mit

Universidad Internacional del Ecuador. (2026). Proyecto Integrador – Tercer Nivel:

Desarrollo de una plataforma empresarial distribuida integrando redes, desarrollo de software y seguridad informática [Guía del proyecto]. UIDE.